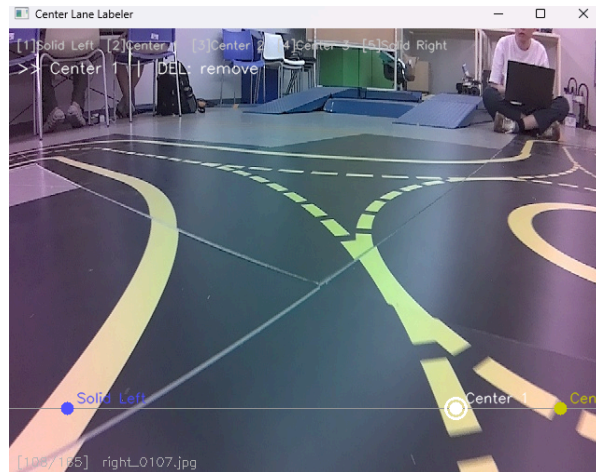


자동차 임베디드 AI 중간 보고서

이름	김민성		
학번	2021011803	학과	미래자동차공학과
Task 1 ~ 4 구현			
기본 제어기 구현	[설계] - 인지, 판단을 검증하기 위해서는 제어가 가능해야 하기 때문에 offset 기반의 간단한 P제어를 구현 - 이후 분업을 통해 다른 팀원이 PID 제어로 발전시킴 [결과] <pre>intensity = max(INTENSITY_MIN, min(INTENSITY_MAX, total)) if offset_ratio > 0: return intensity, INNER_SPEED else: return INNER_SPEED, intensity</pre> - P 제어 부분은 offset에 비례항을 곱하여 total을 계산하는 형태 - offset이 너무 작은 경우 돌아가지 않는 문제가 있어 MIN, MAX 로 제한을 둠 - INNER_SPEED를 고정하여 비선형적으로 회전하는 특성을 억제		
차선 인지 모델 학습	[설계] - 수업시간에 배운 차선 인지 모델을 확장한 'N개 차선 모델'로 결정 - 모든 Task에서 3개의 선만 잘 구분할 수 있다면 차선을 따라 제어하는 데에는 문제가 없다고 생각하였음 - 특히 로타리에서는 두개의 차선을 잘 인지하는 경우 출구를 선택할 수 있다는 장점 [라벨링] - 팀원이 수집한 데이터를 이용해 모든 점선, 실선에 순서를 포함하여 라벨링 - 점을 키보드로 선택하고 찍을 수 있는 cv2 라벨링 툴을 제작하여 600장 가량을 라벨링을 약 2시간 이내에 끝냄		



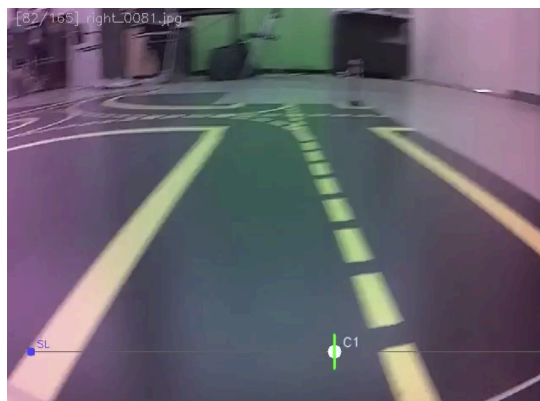
(라벨링 툴 화면)

[학습]

- mobilenetv2, 100 epoch을 고정한 후 다른 파라미터를 변경하며 학습을 진행
- 처음 300장을 라벨링 후 학습한 결과 튜닝을 해도 정확도가 떨어졌음
- 이후 300장을 추가해 총 600장을 학습하여 충분한 정확도를 확보

[결과]

동일 장면의 정확도 비교 장면



(300장 학습, 정확도 낮음)



(600장, 정확도 향상)

[한계]

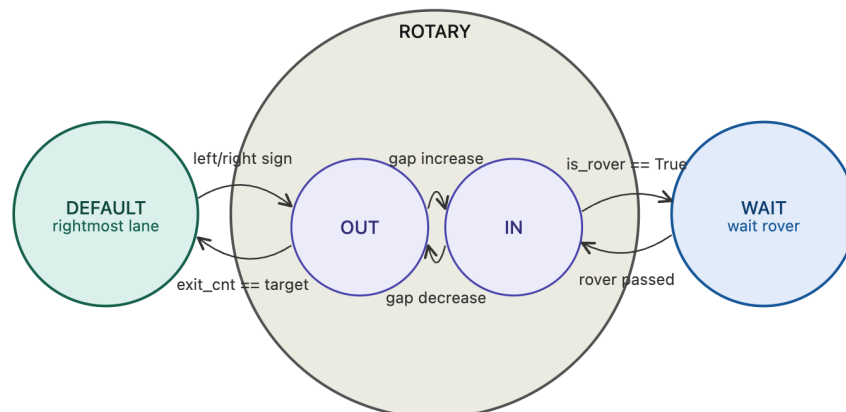
- 3개의 차선을 인지하기에는 학습 데이터가 부족하여 신호등에서는 직진을 강제하는 로직을 넣어주었음
- 카메라의 각도가 틀어지는 경우 차선 인지 정확도가 크게 떨어짐
(카메라 문제인지 알아차리지 못해 시간 낭비를 많이 하였음)

[기본 로타리 로직]

- IN, OUT을 체크하여 바퀴 수를 세팅하는 형태로 구현
- 처음 ROTARY state에 진입할 때부터 출구 타겟의 세팅이 필요
- 의외로 2개의 차선을 안정적으로 감지하여, 빠르게 튜닝할 수 있었음



로타리 판단 로직
구현 및 튜닝



[로버 로직]

1. 첫 IN이 인지되기 전까지 로버를 인식하면 IN에서 WAIT 상태로 변경
2. 로버의 BB 위치가 절반 이상을 지나고, 점점 증가하는 경우 다시 ROTARY로 이동하여 기존 상태에서 이어서 수행

[한계]

- 시연 때 로타리를 탈출하였지만 표지판이 보여 다시 로타리로 들어가는 문제가 발생
- 로타리에 진입한 후, 차량이 가까워지면 정지하는 로직을 넣지 못함

전체 판단 로직 구현	[설계] - 로타리의 카운터 기반 판단이 잘 작동하는 것을 보며 기본 판단 로직 또한 카운터 기반으로 구현을 결정 - 'STOP'는 traverse 시 정지를 중단하는 데에 사용할 타이머의 세팅 필요 [결과] - 지정한 frequency에 맞추어 아래 함수를 실행하며 traverse한다 - threshold 를 dictionary로 구현하여 매우 간단하게 튜닝 가능 <pre> # 각 class별 size가 얼마일 때 counter를 올릴지 CLASS_SIZE_TR = { 'LEFT': 600, 'RIGHT': 600, 'STOP': 2500 } #각 class별 counter가 얼마일 때 traverse할지 COUNTER_TR = { 'LEFT': 10, 'RIGHT': 10, 'STOP': 10 } (counter parameters) def _process_yolo(self, detections): self.detections = detections self._update_class_counters(detections) #detections에 있는 애들 중 만족하는 놈들 counter + or - if self.state == 'DEFAULT': # TR 넘은 애들 찾아서 traverse triggered = self._check_counter() if triggered == 'LEFT': self.exit_target = 4 </pre> (traverse logic)
ROS 시행 착오	앞에서 다루지 않았지만, 전체 구현 중에 가장 많은 시간을 잡아 먹은 것은 ROS를 적용한 후 제어가 제대로 되지 않았던 것이다. ROS에 대한 이해가 부족한 채로 사용하여 각 노드 간의 안정성이 확보되지 않았고, 제어기에 지연이 생기면서 차선을 이탈하는 문제가 발생했다. 카메라 프레임의 불필요한 memory 접근, GPU를 잘 사용하는 문제 등 여러가지 원인을 추측하고 개선해보았지만, 동일한 코드임에도 ROS환경에서만 차선을 이탈하는 문제가 지속적으로 발생하였다. 결국 이틀 정도 후에 파이썬의 스레드를 이용하는 것으로 코드를 전부 리팩터링 하고 시스템이 정상화 되었다. 완전히 이해하지 못한 채로 툴을 사용하면 안된다는 점을 다시금 깨달았다.

Your own problem

[목표]

오프로드 환경에서 목적지까지 안전하게 이동하기

- 꼬깔, 표지판 등 차선 대신 경계를 나타내는 요소를 인식하여 주행 경로를 이탈하지 않는다.
- 장애물을 감지하고 회피한다.
- 지정된 주차 구역이 있는 경우 해당 구역 내에 정확히 정차한다.

[선정 이유]

기존 프로젝트에서 사용한 차선은 비교적 명확하여, y축을 고정한 센터라인 모델로 추정이 가능했다. 그러나 실제 상황에서는 차선이 가리거나, 공사 중에는 꼬깔로 차선을 표시하는 등의 예외 상황이 존재한다. 기존 모델은 이런 상황에서 정상적인 주행이 불가능하다는 한계가 존재하므로 목표를 선정하였다.